

Assignment CA_ONE Report

PROGRAMMING FOR DATA ANALYTICS (Course Code: B9DA108)

Academic & Student Details

Data Attribute	Information
Submitted To	Alexander Victor
Submitted By	Saif Uddin Vuia
Student Id	20102237
Program	MSc in Data Analytics
Institution	Dublin Business School

Assignment Code Access Links

- **Google Colab Notebook:**
  Programming_Assig-01.ipynb
- **GitHub Repository:**
  https://github.com/saifuddin-vuia/saifuddin-vuia/blob/main/Programming_Assig_01.ipynb

Table of Contents

Academic & Student Details **1**
Assignment Code Access Links **1**
Table of Contents **2**
Overview of the Report **3**
Part A: House Hunting **3**
Problem Description **3**
AI_Assisted Python Code 4
Student_Python Code 4
Discussion of Differences 4
Part B: Savings, with a Raise **5**
Problem Description 5
AI_Assisted Python Code 5
Student_Python Code 6
Discussion of Differences 6
Part C: Savings, with a Raise **6**
Problem Description 6
Search Algorithm for Bisection 7
AI_Assisted Python Code 7
Student_Python Code 8
Discussion of Difference 9
Test cases & recalculated results **10**
Reflection & Learning 10
Critical Evaluation & Reflection **10**
Algorithmic Efficiency 10
Practical Limitations of Finance Assumptions 11
AI Framework Declaration with References **11**

Overview of the Report

The following report illustrates a sequential coding project on modeling personal savings for the **B9DA108: Programming for Data Analysis course (CA One).** The project examines the amount of time taken to save enough funds for buying a house in Galway, Ireland. It does so by conducting tests using various levels of savings, interest rates, and income growth:

- **Part A**: Calculates the initial savings over a period based on a constant income.
- **Part B:** Adds salary increases every six months to see how career growth speeds up savings.
- **Part C:** It makes use of an intelligent search algorithm known as **Bisection Search** in determining the precise percentage of the salary that one requires for savings to meet the home deposit target in 36 months, with an accuracy of $100.

The report makes a comparison between the code written with the help of AI and the code written by me for Parts A, B & C. This comparison focuses on the structures of the code, the way calculations are done, the outputs generated by the programs, and the usage of Python programming concepts.

Part A: House Hunting

Problem Description

In Part A, I figure out how many months it takes to save a 25% down payment for a home.

AI_Assisted Python Code

AI Code for Part A

Prompt Used:"Develop a python code to compute the time is required to sav efor a 25% down paymnet towards dream house based on annual income, saving percentage, house price and 4% interest rate per year."

```
# Get user inputs for salary, savings percentage, and house cost
annual_salary = float(input("Enter your annual salary: "))
portion_saved = float(input("Enter the percent of your salary to save, as a decimal: "))
total_cost = float(input("Enter the cost of your dream home: "))

# Fixed initial values and parameters
portion_down_payment = 0.25  # 25% down payment requirement
down_payment = total_cost * portion_down_payment
current_savings = 0.0
r = 0.04  # Annual return on investment (4%)
months = 0

# Loop until savings reach the down payment target
while current_savings < down_payment:
    # Add monthly investment return
    current_savings += current_savings * (r / 12)
    # Add monthly savings from salary
    current_savings += (annual_salary / 12) * portion_saved
    # Track elapsed months
    months += 1

print("Number of months:", months)
```

Student_Python Code

Student Solution

```
# Initial User Inputs
annual_income = float(input("Enter user annual salary: "))
savings_pct = float(input("Enter the salary saving percentage (as decimal): "))
house_cost = float(input("Enter dream home cost: "))

required_deposit = house_cost * 0.25
total_saved = 0.0
monthly_income = annual_income / 12.0
r = 0.04
month_count = 0

# Savings month by month until target is reached
while total_saved < required_deposit:
    total_saved += total_saved * (r / 12.0)
    total_saved += monthly_income * savings_pct
    month_count += 1

print(f"It will take {month_count} months to save for a down payment.")
```

Discussion of Differences

The key difference in the calculation of the monthly salary between AI & my code is that in AI, the annual salary is divided by 12 in the while loop, while in my code the monthly income is calculated before the while loop. AI uses a different variable for calculating the percentage of the down payment, while I used 25% of the cost of the house in my code. Also, my code uses an f-string for showing the output. There is no input validation in either code, so it might cause an infinite loop if the saving rate is 0 and the deposit is positive. Despite different code structures, the two programs solve the same problem.

Part B: Savings, with a Raise

Problem Description

In Part B, modify the savings model to account for salary growth. The user inputs a raise percentage, and every six months, the annual salary grows at that rate.

AI_Assisted Python Code

AI Code for Part B

Prompt Used:"MOdify the python code for savings such that it takes into account a percentage increase in semi-annual raises for annual income."

```python
# Get user inputs including the semi-annual raise
annual_salary = float(input("Enter your starting annual salary: "))
portion_saved = float(input("Enter the percent of your salary to save, as a decimal: "))
total_cost = float(input("Enter the cost of your dream home: "))
semi_annual_raise = float(input("Enter the semi-annual raise, as a decimal: "))

# Fixed initial values and parameters
portion_down_payment = 0.25
down_payment = total_cost * portion_down_payment
current_savings = 0.0
r = 0.04
months = 0

# Loop until savings reach the down payment target
while current_savings < down_payment:
    # Add monthly investment return
    current_savings += current_savings * (r / 12)
    # Add monthly savings from salary
    current_savings += (annual_salary / 12) * portion_saved
    # Track elapsed months
    months += 1

    # Increase annual salary every 6 months
    if months % 6 == 0:
        annual_salary += annual_salary * semi_annual_raise

print("Number of months:", months)
```

Student_Python Code

Student Solution

```
# Inputs including semi-annual raise rate
annual_income = float(input("Enter user annual salary: "))
savings_pct = float(input("Enter the salary saving percentage (as decimal): "))
house_cost = float(input("Enter dream home cost: "))
raise_pct = float(input("Enter the semi-annual raise, (as decimal): "))

required_deposit = house_cost * 0.25
total_saved = 0
monthly_income = annual_income / 12
r = 0.04
month_count = 0

while total_saved < required_deposit:
    #Monthly investment return first
    total_saved += total_saved * (r / 12)
    # Monthly salary contribution
    total_saved += monthly_income * savings_pct
    month_count += 1

    # Apply salary bump every 6 months
    if month_count % 6 == 0:
      monthly_income += monthly_income*raise_pct

print(f"It will take {month_count} months to save for down payment.")
```

Discussion of Differences

The main difference between the two programs is that in the AI code, the annual salary is kept, while in my program, the monthly salary is stored in a variable called monthly_income. The AI version & my code increase the annual salary every six months. Both programs use the same interest calculation and check for input values to avoid entering invalid entries and getting into an infinite loop.

Part C: Savings, with a Raise

Problem Description

Part C covers a specific financial question: What percentage of user salary must the user save to reach a $250,000 deposit on a $1M home in 36 months?

- ➔ Timeframe: 36 months
- ➔ Semi-annual rise: 7%
- ➔ Annual return: 4%
- ➔ Accuracy target: Within $100 of $250,000 (epsilon = 100)

Search Algorithm for Bisection

In order to prevent the problem of endless computation due to decimal figures, an integer-based search is applied:

- **Search Space:** Integers from the range of 0 to 10,000 (representing percentages of savings from 0.00% to 100.00% are used).
- **Search Approach:** A binary search is used to narrow down the search space.
- **Calculation of Percentage:** Every guess in integers is divided by 10,000 and used as the rate (rate =guess / 10,000).
- **Testing:** Test is done for 100% savings rate (10,000). If the savings possible in 36 months do not go above $249,900, the program prints out the message that this goal is not attainable in 3 years given this salary.

AI_Assisted Python Code

AI Code for Part C

Prompt Used:"Implement a Python code that uses besection search to determine the savings rate required to make a 250,000 down paymnet in 36 months on a house worth 1,000,000 given 7% semi-annual increments and an annual return of 4% on investments."

```python
# Get starting salary input
starting_salary = float(input("Enter the starting salary: "))

# Fixed assignment constants
total_cost = 1000000.0  # $1,000,000 home
down_payment = total_cost * 0.25  # $250,000 down payment
semi_annual_raise = 0.07  # 7% raise every 6 months
r = 0.04  # 4% annual return
target_months = 36  # Target savings period of 3 years

# Helper function to calculate total savings after 36 months for a given rate
def calculate_savings(rate_integer, salary):
    portion_saved = rate_integer / 10000.0
    current_savings = 0.0
    annual_salary = salary

    for month in range(1, target_months + 1):
        current_savings += current_savings * (r / 12)
        current_savings += (annual_salary / 12) * portion_saved
        if month % 6 == 0:
            annual_salary += annual_salary * semi_annual_raise

    return current_savings

# Verify if saving 100% (10000) is enough to reach the target within $100
if calculate_savings(10000, starting_salary) < (down_payment - 100):
    print("It is not possible to pay the down payment in three years.")
```

```
else:
    # Set search boundaries (0% to 100% mapped to 0-10000)
    low = 0
    high = 10000
    steps = 0
    found = False

    # Perform bisection search
    while low <= high:
        steps += 1
        guess = (low + high) // 2
        savings = calculate_savings(guess, starting_salary)

        # Check if savings fall within $100 of required down payment
        if abs(savings - down_payment) <= 100:
            found = True
            break
        elif savings < down_payment:
            low = guess + 1
        else:
            high = guess - 1

    if found:
        best_rate = guess / 10000.0
        print("Best savings rate:", best_rate)
        print("Steps in bisection search:", steps)
```

Student_Python Code

Student Solution

```
# Inputs for the calculation
annual_income = float(input("Enter user annual salary: "))
house_cost = 1000000
raise_percent = .07
margin_of_error = 100
deposit_goal = house_cost * 0.25  # 25% down payment goal ($250,000)
timeframe_months = 36

def simulate_savings(pct_saved):
    savings_balance = 0
    r = 0.04
    current_monthly_pay = annual_income / 12

    for month in range(1, timeframe_months + 1):
        # Monthly interest first
        savings_balance += savings_balance * (r / 12)
        # Monthly savings from salary
        savings_balance += current_monthly_pay * pct_saved

        # Salary bump every 6 months
        if month % 6 == 0:
            current_monthly_pay += current_monthly_pay*raise_percent

    return savings_balance

if simulate_savings(1.0) < (deposit_goal - margin_of_error):
    print("It is not possible to pay the down payment in three years.")
```

```
else:
        low_bound = 0
        high_bound = 10000
        bisection_steps = 0
        found = False

        # Binary search to find optimal rate (between 0 to 10,000)
        while low_bound <= high_bound:
            bisection_steps += 1
            guess_rate = (low_bound+high_bound) // 2
            decimal_val = guess_rate / 10000
            total_saved = simulate_savings(decimal_val)

            if abs(total_saved - deposit_goal) <= margin_of_error:
                found = True
                break

            elif total_saved < deposit_goal:
                low_bound = guess_rate + 1

            else:
                high_bound = guess_rate - 1
        if found:
            print(f"Optimal savings rate: {guess_rate / 10000}")
            print(f"Bisection search steps taken: {bisection_steps}")

        else:
            print(f"No saving rate was found within the required toleranc")
```

Discussion of Difference

The most notable difference in the function design is that the AI calculate_savings() takes salary as its argument, while my program's calculation_saving() uses an external variable. However, differences in the design of the function, the range of searching, and the terminating conditions may affect the manner in which the search is carried out, as well as the number of steps taken. The AI function takes care of converting the saving rate to a decimal inside the function itself, whereas my program does so before calling the function. Both programs use bisection search in 36 months with a $100 tolerance level. Also, my program makes sure that the saving rate is converted to a decimal before being called in the function, and uses descriptive variables along with f-strings.

Test cases & recalculated results

Part / Test	Inputs	My version	Gemini version
A / 1	Salary 120,000; save 0.1; house 500,000	105 months	105 months
A / 2	Salary 135,000; save 0.15; house 4,500,000	352 months	352 months
B / 1	Salary 120,000; save 0.05; house 500,000; raise 0.3	59 months	59 months
B / 2	Salary 140,000; save 0.08; house 6,000,000; raise 0.5	72 months	72 months
C / 1	Starting Salary 150,000	Rate 0.441; 12 steps	Rate 0.441; 12 steps
C / 2	Starting Salary 125,000	Rate 0.5292; 9 steps	Rate 0.5292; 9 steps
C / 3	Starting Salary 0	Impossible message	Impossible message

Reflection & Learning

The comparative study between the coding done by AI versus my own helped me understand the way in which identical programming problems could be solved using different coding approaches. This also helped me realize the effect that savings & salary increase had on the calculations involved in the problem, as well as how the bisection search approach could help reduce the range in order to determine the right amount of savings rate. What also became clear to me is that creating output isn't enough; the program has to be validated.

Critical Evaluation & Reflection

Algorithmic Efficiency

In Part A and Part B, there is an implementation of linear sequential simulation loops (while loops) for time (N months) as an unknown variable that depends on accumulation. Also, in Part C, finding an optimal savings rate using an exhaustive linear search would require up to 10,000 interactions in the worst-case scenario. By converting the search interval into a discrete integer space (0 to 10,000) and applying **Bisection Search,** the time complexity is reduced to O(log2 N). Because log2(10000) ≈ 13.28, the bisection search algorithm is sure to find the optimal solution within a maximum of **14 steps,** dramatically improving execution performance.

Practical Limitations of Finance Assumptions

- **Volatile Investment:** The model assumes a fixed annual return rate; in real-world markets, the investment rate is subject to volatility.
- **Fixed Inflation & House Price:** Real estate prices in Galway are assumed constant over time. In reality, housing price inflation might outpace savings rates.
- **Tax Considerations:** The models treat gross salary and investment yields as net income without accounting for income tax or capital gains tax.

AI Framework Declaration with References

In accordance with Dublin Business School policy regarding Generative Artificial Intelligence, this submission adheres to **AI Task Completion, Human Evaluation.**

- **AI Applications:** Generative AI was employed as an interactive programmer's co-pilot to aid in correcting syntax errors within control flows, understanding bisection search boundary conditions, and structuring my Markdown report. Like(ChatGPT, Gemini, YouTube & Grammarly).
- **Human Evaluation & Integrity:** All code constructs, variable bindings, and algorithmic logic were manually reviewed, tested against sample test cases, and validated for complete alignment with the CA1 assessment.